

인턴 · 초급 개발자 강의자료

컨테이너 · Docker 그리고 쿠버네티스 입문

"제 컴퓨터에선 되는데요?"에서 시작해 대규모 자동 운영까지 — 30
분에 잡는 큰 그림

컨테이너란?

VM vs 컨테이너

Docker 핵심

기술 발전사

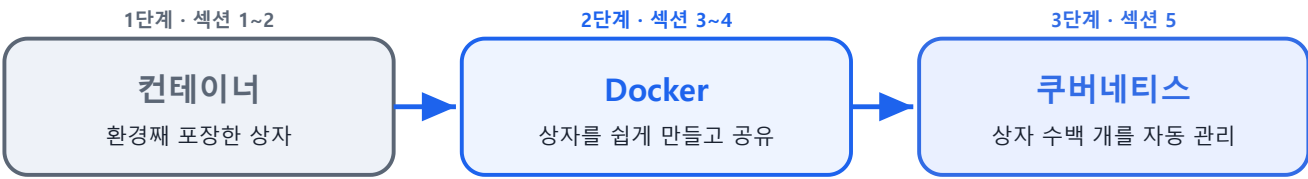
쿠버네티스 기초

한 줄 요약 · 컨테이너는 '환경째 포장', Docker는 '쉽게 만들고 공유', 쿠버네티스는 '많은 컨테이너 자동 관리'

시작하며 — 오늘의 지도

왜 지금 배우나

요즘 실무 서버 배포는 대부분 **컨테이너** 위에서 돌아갑니다. 클라우드(AWS·GCP·네이버클라우드 등), 마이크로서비스(MSA), CI/CD 자동배포가 모두 이 기술을 바탕에 깔고 있어, **Docker·쿠버네티스는 백엔드/데브옵스 채용공고의 단골 키워드**입니다. 오늘은 이 두 가지의 **큰 그림과 '왜'를 잡는 것**이 목표입니다. (명령어 암기·실습은 다음 기회에)



하나의 예제로 꿰기

낯선 용어가 쏟아져도 길을 잃지 않도록, 여러분이 만든 `app.jar` (스프링부트 앱) 하나를 처음부터 끝까지 따라갑니다.

내 `app.jar` → Docker로 이미지 만들기 → 레지스트리에 push → 쿠버네티스가 pull해서 파드로 실행

이 한 줄이 오늘 배울 모든 조각을 연결하는 뼈대입니다. Docker와 쿠버네티스를 잇는 다리가 바로 **'이미지'**라는 점을 기억하세요.

30분 시간 배분 (강사용)

시간	내용	핵심 메시지
6분	1. 컨테이너 & Docker 등장 — '제 컴퓨터에선 되는데' 흑 + 화물 컨테이너 비유	환경째 포장
4분	2. VM vs 컨테이너 — 아파트 비유, 커널 공유	게스트 OS가 없다
8분	3. Docker 핵심 — 이미지/컨테이너(붕어빵), build→push/pull→run	실무 직결(최대 비중)
3분	4. 발전사 — 4단계 아크만 빠르게	진화의 결과
7분	5. 쿠버네티스 — 왜 필요 + 5개념 + 선언형	대규모 자동 관리
2분	마무리 — 기억할 3가지 + Q&A	수미상관으로 닫기

※ 우선순위: Docker(실습 직결)에 최대, 발전사에 최소, 쿠버네티스는 '이런 게 있다' 인지 수준까지.

1 컨테이너란 무엇인가 & Docker의 등장

한 줄 결론 먼저

컨테이너(Container)는 "내가 만든 프로그램 + 그 프로그램이 돌아가는 데 필요한 모든 것(라이브러리, 설정, 실행 환경)"을 하나의 상자에 담아, 어떤 컴퓨터에서도 거의 똑같이 실행되게 만든 격리된 실행 단위입니다. 그리고 **Docker(도커)** 는 2013년 무렵 등장해 이 컨테이너를 **누구나 쉽게** 만들고 공유하게 해 준 도구입니다. 오늘 배울 컨테이너 세계의 출발점이 바로 이 두 가지입니다.

1. "제 컴퓨터에선 되는데요?" — 모두가 겪는 그 문제

개발을 하다 보면 반드시 한 번은 듣거나 말하게 되는 문장이 있습니다.

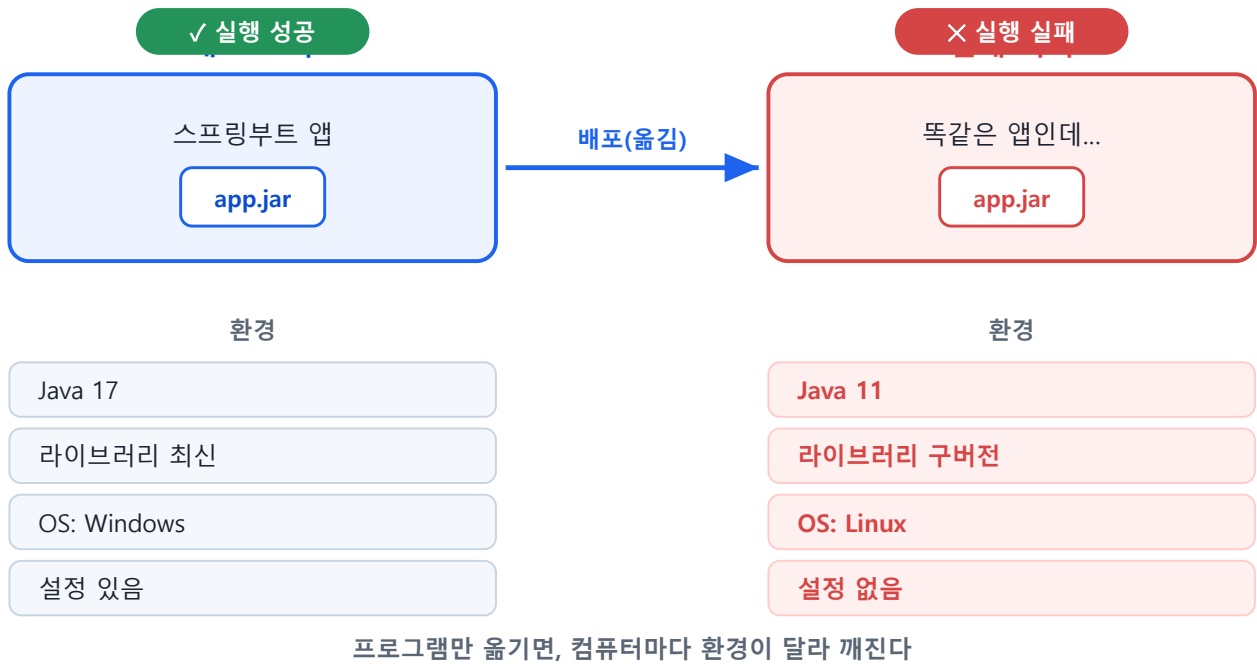
참고

"이상하네요, 제 컴퓨터에선 분명히 잘 됐는데요?"

내 노트북에서는 스프링부트 앱이 잘 실행됐는데, 옆자리 동료 PC나 회사 서버에 올리면 에러가 터집니다. 왜 이런 일이 생길까요? **컴퓨터마다 "환경"이 다르기 때문**입니다. 대표적인 차이는 이런 것들입니다.

환경 차이 요소	내 노트북	실제 서버
자바(Java) 버전	17	11
라이브러리 버전	최신	예전 버전
운영체제(OS)	윈도우(Windows)	리눅스(Linux)
환경변수·설정 파일	내가 세팅해 둬	비어 있음

문제는 이 차이를 사람이 매번 눈으로 맞추기가 거의 불가능하다는 점입니다. 어제 되던 게 오늘 안 되고, 배포할 때마다 "이번엔 또 뭐가 다르지?"를 찾느라 시간을 씁니다. **컨테이너는 바로 이 "환경 차이"로 생기는 배포 사고를 크게 줄이기 위해 나온 해결책**입니다.



2. 컨테이너의 핵심 아이디어 — "환경째로 통째로 포장한다"

기존 방식은 "프로그램만" 서버에 옮겼습니다. 그러니 옮겨간 곳의 환경이 다르면 안 됩니다. 컨테이너의 발상은 정반대입니다.

프로그램만 옮기지 말고, 그 프로그램이 돌아가던 "환경 전체"를 함께 상자에 담아 통째로 옮기자.

이 상자 안에는 내 앱(예: `app.jar`)뿐 아니라, 그 앱이 필요로 하는 **자바 실행 환경, 라이브러리, 설정까지** 전부 들어 있습니다. 그래서 이 상자는 내 노트북에서든, 테스트 서버에서든, 클라우드에서든 **매번 똑같은 내용물로 똑같이 실행**됩니다. 실행되는 컴퓨터에 무엇이 깔려 있는지에 거의 좌우되지 않습니다.

또 하나 중요한 성질은 **격리(Isolation, 서로 분리됨)**입니다. 각 컨테이너는 자기만의 상자 안에서 돌아가므로, 옆 컨테이너와 라이브러리 버전이 충돌하거나 서로의 파일을 건드리는 일이 없습니다. 한 서버 위에 A앱 상자, B앱 상자를 나란히 올려도 내용물이 뒤섞이지 않습니다.

참고

참고: 컨테이너는 **하나의 운영체제(OS)를 여러 상자가 나눠 쓰는** 방식이어서 가상머신(VM)보다 훨씬 가볍습니다. 대신 하드웨어까지 통째로 분리하는 VM보다 격리는 조금 약합니다 — 자원(CPU·메모리) 사용 한도를 따로 정해 두지 않으면 한 상자가 자원을 독차지해 옆 상자를 느리게 만들 수도 있습니다. 그래서 여기서 말하는 격리는 '완벽한 벽'이라기보다 '**프로세스·자원 수준의 분리**'로 이해하는 편이 정확합니다. '가벼운 대신 격리는 조금 느슨한' 이 맞바꿈(트레이드오프)의 자세한 이유는 다음 섹션 "**컨테이너 vs 가상머신**"에서 다룹니다. 여기서는 "환경째로 포장하는, VM보다 가벼운 상자" 정도로만 기억하면 충분합니다.

3. 비유로 이해하기 — 화물 컨테이너 이야기

컨테이너라는 이름은 우연이 아니라 **실제 화물 컨테이너**에서 따온 것입니다. 이 비유 하나면 개념 전체가 이해됩니다.

화물 컨테이너 이전의 항구를 상상해 봅시다. 예전에는 짐마다 모양이 제각각이었습니다. 자루에 담긴 곡물, 나무 상자에 든 기계 부품, 통에 든 기름... 배에 실을 때마다 짐 모양에 맞춰 일일이 다르게 쌓아야 했고, 배에서 트럭으로 옮길 때 또 손이 갔습니다. 느리고, 실수가 잦았습니다. **이게 바로 "프로그램만 옮기던 옛날 배포"의 모습입니다.**

그러다 **"표준 규격의 쇠 상자(컨테이너)"가 등장합니다.** 안에 무엇을 담은 겉모습과 크기는 똑같습니다. 그러자 마법 같은 일이 벌어집니다.

- 짐이 무엇이든 **배, 트럭, 기차 어디에나 똑같은 방식으로** 실을 수 있습니다.
- 항구의 크레인도 안에 뭐가 들었는지 몰라도 됩니다. **"표준 상자"라는 것만 알면** 척척 옮깁니다.

소프트웨어 컨테이너도 똑같습니다. 안에 담긴 게 스프링부트 앱이든 파이썬 앱이든, **겉의 규격(컨테이너)이 표준이라서** 내 노트북·테스트 서버·클라우드 어디서나 똑같은 방식으로 실행됩니다. "안에 뭐가 들었는지 신경 쓰지 않고 규격만 맞으면 어디든 옮긴다" — 이것이 컨테이너의 힘입니다.

한 가지 용어만 더 붙이면, **이미지(Image)**는 **"붕어빵 틀"**이고 **컨테이너**는 **"그 틀로 찍어낸 붕어빵"**입니다. 같은 틀(이미지) 하나로 똑같은 붕어빵(컨테이너)을 몇 개든 찍어낼 수 있습니다. 이미지의 자세한 구조는 뒤에서 다시 다룹니다.

현실 세계 — 표준 화물 컨테이너



소프트웨어 세계 — 컨테이너(환경패 포장)



4. Docker의 등장(2013년 무렵) — "누구나 쉽게"를 실현하다

여기서 흔한 오해 하나를 바로잡고 갑니다. **컨테이너 기술 자체는 Docker가 처음 발명한 것이 아닙니다.** 리눅스에는 이미 프로세스를 격리하고 자원을 통제하는 기술(네임스페이스, cgroups, LXC 등)이 존재했습니다. 다만 이 기술들은 **전문가만 다룰 수 있을 만큼 복잡하고 손이 많이 갔습니다.**

참고

용어 각주(외울 필요는 없습니다): 정확히는 역할이 나뉩니다. **네임스페이스(namespace)**는 '각 상자가 무엇을 볼 수 있는가'를 갈라 주는 **격리**를 맡고, **cgroups(control groups)**는 'CPU·메모리를 얼마나 쓸 수 있는가'를 정하는 **자원 제어**를 맡습니다. 흔히 둘을 뭉뚱그려 "격리 기술"이라 부르지만, 자원을 나누는 쪽은 cgroups라는 점만 구분해 두면 충분합니다.

Docker가 한 일은 발명이 아니라 **대중화**였습니다. 2013년 무렵 공개된 Docker는 이 어려운 컨테이너 기술을 "**명령어 몇 줄**"과 "**짧은 설정 파일**" 수준으로 끌어내렸습니다. 예를 들어 스프링부트 앱을 컨테이너로 포장하는 설명서(Dockerfile)는 이 정도로 짧습니다.

```
# 예시일 뿐 - 자세한 문법은 뒤 섹션에서
FROM eclipse-temurin:17-jre # 자바 실행 환경을 통째로 가져온다
COPY app.jar /app.jar      # 내 스프링부트 앱을 상자에 넣는다
CMD ["java","-jar","/app.jar"] # 상자가 실행될 때 이 명령을 돌린다
```

이 짧은 설명서 하나로 "자바 환경 + 내 앱"이 담긴 표준 상자가 만들어집니다. (예시의 `eclipse-temurin`은 여러 공식 자바 이미지 중 하나이며, `amazoncorretto` 같은 다른 공식 이미지로도 똑같이 만들 수 있습니다.)

5. 왜 폭발적으로 인기였나 — 세 가지 이유

Docker가 순식간에 표준이 된 이유를 중요도 순으로 정리하면 다음과 같습니다.

1. **압도적으로 쉬웠다.** 전문가의 영역이던 컨테이너를, 신입 개발자도 하루면 흉내 낼 수 있게 만들었습니다. 진입 장벽이 무너진 것이 가장 큰 힘이었습니다.
2. **"내 컴퓨터에선 되는데요" 문제를 크게 줄였다.** 개발 → 테스트 → 운영 서버까지 **똑같은 상자**가 흘러가니, 환경 차이로 인한 사고가 크게 줄었습니다. 개발자와 운영자 사이의 다툼도 줄었습니다.
3. **쉽게 공유할 수 있었다.** 내가 만든 상자(이미지)를 공개 저장소에 올려 두면, 남들은 명령어 한 줄로 그대로 내려받아 똑같이 실행할 수 있습니다. "잘 만든 환경을 그대로 나눠 쓴다"는 문화가 생겼습니다.

참고

솔직한 한 줄: 컨테이너가 환경 차이를 '0'으로 만들지는 못합니다. CPU 아키텍처(예: 인텔 계열 amd64 vs 애플 실리콘 일부 서버의 arm64)나 호스트의 커널·드라이버 차이는 여전히 남습니다. 그래도 "어떤 자바·어떤 라이브러리·어떤 설정으로 돌리는가" 같은 **사고의 주범 대부분을 상자 안에 고정**해 주기 때문에, 실무 체감으로는 배포 사고가 확 줄어듭니다.

정리하면, Docker는 새로운 기술을 발명했다기보다 "**어려운 기술을 누구나 쓸 수 있게, 그리고 서로 나눌 수 있게**" 만들었기 때문에 인기였습니다. 그리고 이렇게 수많은 컨테이너가 생겨나자, 자연스럽게 "이 많은 상자들을 어떻게 자동으로 관리하지?"라는 다음 고민이 등장합니다. 그 답이 뒤에서 배울 **쿠버네티스(Kubernetes)**입니다.

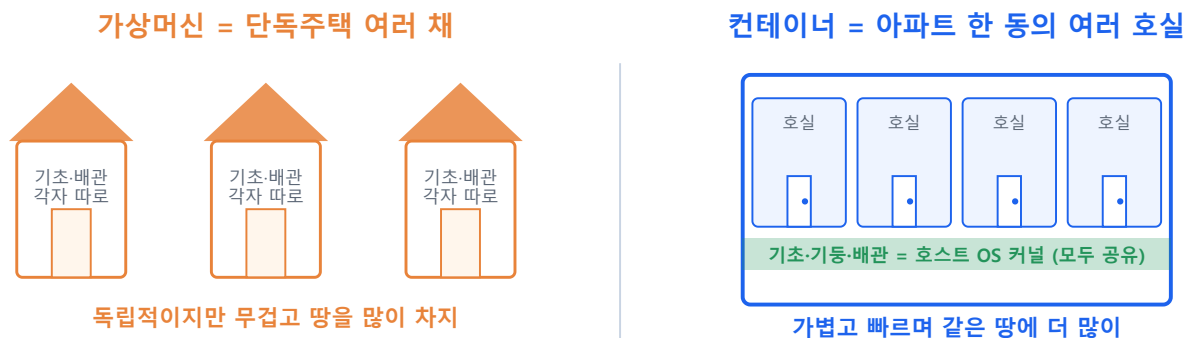
2 가상머신(VM) vs 컨테이너

결론부터: 컨테이너가 VM보다 가볍고 빠른 이유는 단 하나입니다. 게스트 OS(각 가상 컴퓨터마다 새로 설치하는 운영체제)를 통째로 올리지 않고, 호스트 OS의 커널(운영체제의 핵심 부분)을 함께 나눠 쓰기 때문입니다. 이 한 문장만 붙잡으면 나머지 차이(무게·부팅 속도·자원 효율)는 전부 여기서 파생됩니다.

한 컷 비유: 집 한 채 vs 아파트 한 호실

- **가상머신(VM) = 단독주택 한 채.** 땅(하드웨어)을 나눠 그 위에 집을 통째로 새로 짓습니다. 기초 공사, 상하수도, 전기 배선까지 집집마다 따로 있습니다. 완전히 독립적이라 옆집과 섞일 일이 없지만, **짓는데 오래 걸리고 무겁고 땅도 많이 차지합니다.**
- **컨테이너 = 아파트 한 호실.** 건물의 기초·기둥·상하수도 배관(호스트 OS의 커널)은 **모두가 공유**하고, 각 호실은 자기만의 현관문과 잠금장치(격리된 공간)만 가집니다. 그래서 **입주가 빠르고 가볍고, 같은 땅에 훨씬 많은 세대가 들어갑니다.** 다만 건물 기초에 문제가 생기면 전 세대가 영향을 받습니다(= 공유 커널의 약점).

여기서 "집을 통째로 새로 짓느냐(게스트 OS를 올리느냐)"가 핵심 갈림길입니다.



각 호실은 현관문·자물쇠로 격리(프로세스 격리)
단, 건물 기초에 문제가 생기면 전 세대 영향 = 공유 커널의 약점

아키텍처로 보면

VM 구조 (무겁다)

하드웨어 → 하이퍼바이저(Hypervisor) → VM마다 [게스트 OS 전체 + 라이브러리 + 앱]

하이퍼바이저는 하드웨어를 잘게 나눠 가짜 컴퓨터(가상 하드웨어)를 여러 대 만들어 주는 소프트웨어입니다. 그 위에 VM마다 리눅스·윈도우 같은 **운영체제를 통째로 새로 설치**합니다. 컴퓨터 안에 컴퓨터를 여러 대 켜 두는 셈이라, OS가 켜지는 부팅 과정을 매번 거쳐야 합니다.

참고

참고(가벼운 심화): 하이퍼바이저는 하드웨어 위에서 직접 도는 **Type 1(베어메탈, 예: ESXi-KVM-Hyper-V)** 과, 이미 깔린 호스트 OS 위에서 도는 **Type 2(예: VirtualBox-VMware Workstation)** 로 나뉩니다. 위 그림은 Type 1 모델이고, 여러분이 노트북에 직접 까는 VirtualBox 같은 도구는 Type 2입니다. 소개 단계에서는 "하드웨어를 가상 컴퓨터로 나눈다"만 기억하면 충분합니다.

컨테이너 구조 (가볍다)

하드웨어 → 호스트 OS(커널) → **컨테이너 엔진(예: 도커 엔진)** → 컨테이너마다 [라이브러리 + 앱] (**게스트 OS 없음**)

컨테이너는 이미 켜져 있는 호스트 OS의 커널을 그대로 빌려 쓰고, 그 위에서 **격리된 프로세스로만** 실행됩니다. 이때 "**격리**"와 "**자원 제한**"은 리눅스 커널의 서로 다른 두 기능이 나뉘 있습니다.

- **네임스페이스(namespace)** — **격리 담당**: "너는 네 방 밖은 못 본다" — 프로세스·네트워크·파일 시스템을 서로 안 보이게 칸막이.
- **cgroups(control groups)** — **자원 제한 담당**: "CPU·메모리는 딱 이만큼만 써라" — 사용량을 제한·배분. (격리 수단이 아니라 자원을 나눠 주는 역할입니다.)

앱 실행에 필요한 라이브러리만 담고 OS 커널은 안 들고 오니, 이미지가 작고 실행이 즉각적입니다.

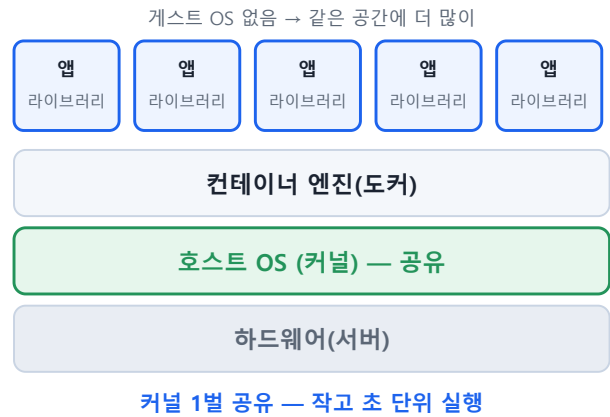
참고

참고: 실무의 **보안** 격리는 이 두 기능만으로 완성되지 않습니다. 시스템 콜을 걸러 내는 **seccomp**, 프로세스 권한을 최소화하는 **capabilities**, **SELinux/AppArmor** 같은 보안 모듈이 함께 작동합니다. 지금은 "네임스페이스=격리, cgroups=자원 제한" 두 가지만 기억하면 충분합니다.

가상머신(VM) — 무겁다



컨테이너 — 가볍다



핵심 차이 = 게스트 OS를 올리느냐

VM은 OS 통째로, 컨테이너는 커널만 공유 — 무게·속도 차이가 여기서 나온다

그래서 왜 가볍고 빠르냐 (정리)

1. **부팅 과정이 없다 (체감상 가장 큰 차이).** 이건 새로운 원인이 아니라 위의 **커널 공유(게스트 OS 미탑재)**에서 곧장 나오는 결과입니다 — 부팅할 OS 자체가 없는 거죠. VM은 켤 때마다 게스트 OS를 부팅해야 해서 대체로 수십 초~분 단위가 걸리지만(환경마다 편차가 큼), 컨테이너는 부팅 없이 프로세스만 뜨므로 보통 **초 단위(때로는 1초 미만)**에 실행됩니다.
2. **크기가 작다.** VM 이미지는 OS를 통째로 담아 대체로 GB 단위입니다. 컨테이너 이미지는 앱+라이브러리만 담아 수 MB~수백 MB 수준인 경우가 많습니다(이미지에 따라 다름).
3. **자원 낭비가 적다.** VM 10대를 켜면 OS도 10벌이 동시에 돌아 메모리·CPU를 중복 소모합니다. 컨테이너는 커널 하나를 공유하므로 같은 하드웨어에 **훨씬 많이(고밀도로)** 띄울 수 있습니다.

그럼 컨테이너가 항상 정답인가? (균형 있게)

아닙니다. **격리 수준과 보안이 중요하다면 VM이, 속도와 밀도가 중요하다면 컨테이너가 유리합니다.**

- **VM이 유리한 경우:** 커널까지 완전히 분리돼 격리·보안 경계가 강함. 서로 다른 OS가 필요할 때(리눅스 위에 윈도우 등), 신뢰하기 어려운 코드를 강하게 가둘 때, 보안 규제가 엄격할 때.
- **컨테이너가 유리한 경우:** 빠른 배포·확장, 마이크로서비스, CI/CD, 개발-운영 환경 일치. 오늘 우리가 배우는 도커·쿠버네티스가 여기에 해당합니다.

여기서 컨테이너의 격리가 "약하다"는 건 **VM에 견준 상대적 표현**이지 무조건 위험하다는 뜻이 아닙니다. 실제로 이 격차는 좁혀지는 추세입니다 — **gVisor**(사용자 공간에서 커널을 흉내 내 한 겹 더 가둠), **Kata Containers**(컨테이너를 경량 VM으로 한 번 더 감쌌) 같은 기술이 공유 커널의 약점을 보완합니다. 여기서는 이름만 알아 두면 됩니다.

실무에서는 **VM 위에 컨테이너를 올려 두 방식을 함께 쓰는 경우가 많습니다.** 클라우드가 대표적입니다 (VM으로 크게 나누고, 그 안에서 컨테이너로 잘게 운영). 관리형 쿠버네티스(EKS·GKE·AKS)의 작업 노드가 VM이고 그 위에서 컨테이너가 도는 구조가 실제 표준입니다.

참고

참고: 컨테이너는 커널을 공유하므로 **리눅스 컨테이너는 리눅스 커널이 필요합니다**. 그래서 윈도우·맥에서 도커를 쓰면 내부적으로 가벼운 **리눅스 VM 한 대**가 숨어서 돌고(윈도우는 WSL2, 맥은 경량 리눅스 VM), 그 위에서 컨테이너가 실행됩니다. "컨테이너가 VM을 완전히 대체한다"기보다 **용도가 다른 도구**라고 이해하면 됩니다.

한눈에 비교

비교 항목	가상머신(VM)	컨테이너(Container)
게스트 OS	있음 (통째로 탑재)	없음 (호스트 커널 공유)
무게 / 이미지 크기	무거움 (대체로 GB 단위)	가벼움 (수 MB~수백 MB)
부팅 / 실행 속도	느림 (수십 초~분)	빠름 (초 단위)
격리 수준	강함 (커널까지 분리)	상대적으로 약함 (커널 공유)
보안 경계	강함	상대적으로 약함 (샌드박스 런타임으로 보완 가능)
자원 효율 / 밀도	낮음	높음
다른 OS 실행	가능	호스트 커널과 같은 계열만*

* "다른 OS 실행"과 "같은 계열"은 모두 **같은 CPU 아키텍처**(예: x86끼리, ARM끼리) 안에서의 이야기입니다. x86 컴퓨터에서 ARM용 OS/이미지를 돌리려면 QEMU 같은 에뮬레이션이 필요해 크게 느려집니다. (요즘 애플 실리콘 맥이 ARM이라 가끔 이미지 호환 문제가 생기는 것도 이 때문입니다.)

3 Docker 핵심 개념과 관련 기술

한 줄 결론: 도커의 핵심은 딱 두 단어 **이미지(image)** 와 **컨테이너(container)** 입니다. "설계도(이미지)를 만들어 → 창고(레지스트리)에 올리고 → 필요한 곳에서 내려받아 → 실행(컨테이너)한다"는 흐름만 잡으면 나머지 기술들은 이 흐름을 뒷받침하는 조연입니다.

1. 이미지 vs 컨테이너 — 봉어빵 틀 vs 봉어빵

가장 헷갈리는 두 단어부터 정리합니다.

- **이미지(image) = 봉어빵 틀.** 실행에 필요한 것(내 프로그램 + 자바 런타임 + 설정)을 통째로 굳혀 놓은 읽기 전용 설계도입니다. 틀 하나는 변하지 않습니다.
- **컨테이너(container) = 그 틀로 구워낸 봉어빵.** 이미지를 실행한 상태가 컨테이너입니다. 틀 하나(이미지)로 봉어빵(컨테이너)을 1개든 100개든 똑같이 찍어낼 수 있습니다.

핵심 포인트: 이미지는 "정지된 설계도", 컨테이너는 "움직이는 실행 인스턴스"입니다. 그래서 "내 컴퓨터에선 되는데?" 문제가 사라집니다. 봉어빵 틀(이미지)이 같으면 어느 가게에서 구워도 같은 봉어빵이 나오기 때문입니다.

2. Dockerfile — 봉어빵 틀을 만드는 레시피

이미지는 손으로 만들지 않고 **Dockerfile** 이라는 **레시피 문서**로 만듭니다. "어떤 재료 위에, 무엇을 넣고, 실행할 때 무슨 명령을 돌려라"를 몇 줄로 적어두면, 그걸 읽어 이미지를 자동으로 구워줍니다.

```
FROM eclipse-temurin:17-jre      # 자바17(JRE)이 깔린 바탕 이미지 위에서 시작
COPY app.jar /app.jar           # 내 스프링부트 jar 를 복사
CMD ["java","-jar","/app.jar"]  # 실행 시 이 명령을 돌려라
```

참고

바탕 이미지 한 줄 참고: 예전 예제에서 흔히 보이던 `openjdk` 공식 이미지는 2022년 무렵 사실상 중단(deprecated)되었습니다. 지금은 `eclipse-temurin` (예제처럼 실행만 필요하면 `-jre`)이나 `amazoncorretto` 같은 이미지를 쓰는 것이 권장됩니다.

비유: Dockerfile은 레시피, 이 레시피로 굽는 행위가 **빌드(build)**, 그 결과물이 봉어빵 틀(이미지)입니다.

3. 레지스트리(Docker Hub) — 이미지의 깃허브

만든 이미지를 남과 공유하거나 다른 서버에서 쓰려면 **레지스트리(registry)** 라는 저장소에 올립니다. 대표적인 공개 레지스트리가 **Docker Hub**입니다.

- **이미지 저장소 = 소스코드 저장소(깃허브)** 와 완전히 같은 구조입니다.
- `push` = 깃허브에 올리기, `pull` = 깃허브에서 내려받기.
- 남이 만든 검증된 이미지(예: `eclipse-temurin`, `mysql`, `nginx`)를 `pull` 한 번으로 가져와 바로 쓸 수 있습니다.

4. 기본 명령 흐름 — build → push/pull → run

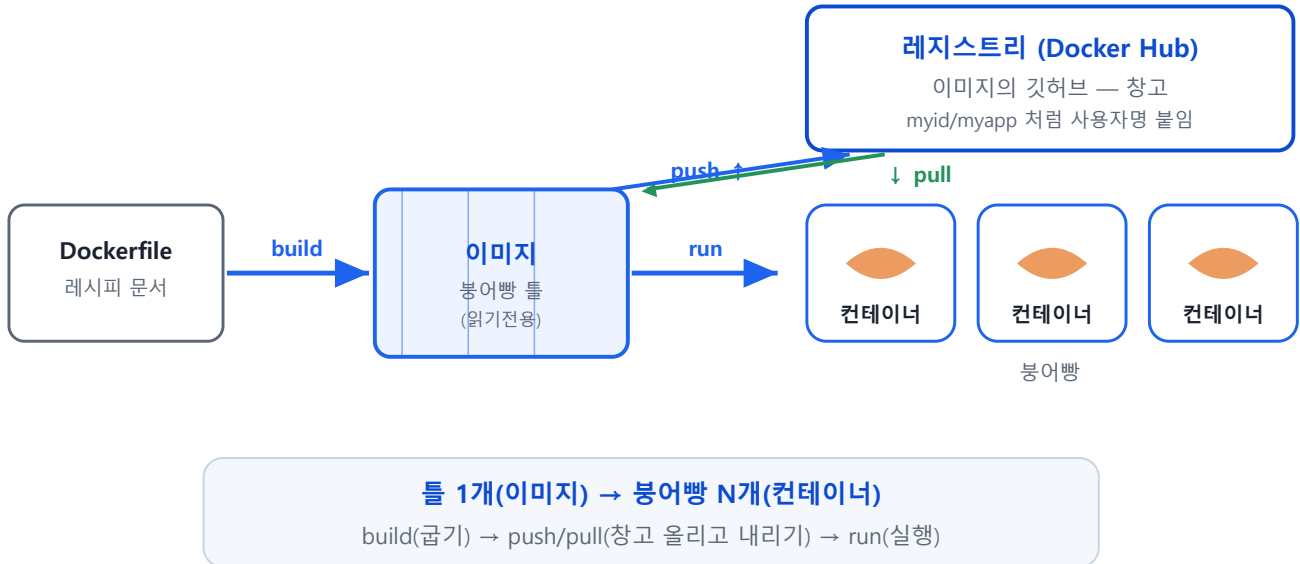
실무에서 반복되는 네 단계입니다. 이 순서만 외우면 됩니다.

1. **build** — Dockerfile로 이미지를 굽는다.
2. **push** — 구운 이미지를 레지스트리(Docker Hub)에 올린다.
3. **pull** — 배포할 서버에서 그 이미지를 내려받는다.
4. **run** — 이미지를 실행해 컨테이너로 띄운다.

```
docker build -t myid/myapp . # 1. 이미지 굽기 (myid = 내 Docker Hub 사용자명)
docker login                # (최초 1회) Docker Hub 로그인
docker push myid/myapp      # 2. 레지스트리에 올리기
docker pull myid/myapp      # 3. 다른 서버에서 내려받기
docker run myid/myapp       # 4. 컨테이너로 실행
```

참고

왜 myapp 이 아니라 myid/myapp 인가: 이미지 이름 앞에 **사용자명(네임스페이스)** 을 붙여야 "내 저장소에 올리는 것"으로 인식됩니다. 이름만 myapp 으로 쓰면 도커 공식 이미지 자리를 가리켜 **올릴 권한이 없어 실패**합니다. 깃허브에서 **내 아이디/저장소이름** 으로 올리는 것과 똑같은 규칙이라고 보면 됩니다.



5. 도커를 떠받치는 리눅스 기술 — namespaces & cgroups

"컨테이너는 어떻게 서로 안 섞이고 독립적으로 돌지?" 그 마법은 사실 **리눅스 커널이 원래 갖고 있던 두 기능** 덕분입니다. 도커가 이걸 편하게 쓰도록 포장했을 뿐입니다.

- **namespaces(네임스페이스) = 격리(칸막이).** 각 컨테이너에 "너만의 방"을 줘서, 자기 프로세스·네트워크·파일시스템만 보이게 합니다. 옆 컨테이너는 존재조차 모릅니다. **비유: 한 사무실을 칸막이로 나눠 각자 독립된 방처럼 쓰는 것.**

- **cgroups(control groups) = 자원 제한.** 각 컨테이너가 쓸 수 있는 **CPU·메모리 양에 상한**을 겁니다. 한 컨테이너가 자원을 독차지해 다른 컨테이너를 굶기지 못하게 막습니다. **비유: 방마다 전기·수도 사용량 한도를 정해두는 것.**

중요: 이 둘은 **리눅스 커널 기능**입니다. 그래서 컨테이너는 근본적으로 리눅스 기술이며, 윈도우·맥에서 도커를 쓸 때는 내부적으로 가벼운 리눅스 환경(맥=경량 리눅스 VM, 윈도우=WSL2 기반 리눅스)을 띄워 그 위에서 돌립니다.

참고

한 줄 예외: 윈도우에는 리눅스 없이 윈도우 커널을 그대로 공유하는 '윈도우 컨테이너' 모드도 따로 있습니다. 다만 실무에서 다루는 컨테이너는 대부분 리눅스 컨테이너라, 이 강의에서는 "컨테이너 = 리눅스 기술"로 이해해도 충분합니다.

6. 관련 기술 지도 (한눈에)

도커 주변 용어들이 많아 보이지만, 각자 맡은 층(layer)이 다를 뿐입니다. 아래 한 줄 정의로 지도를 그려 두세요.

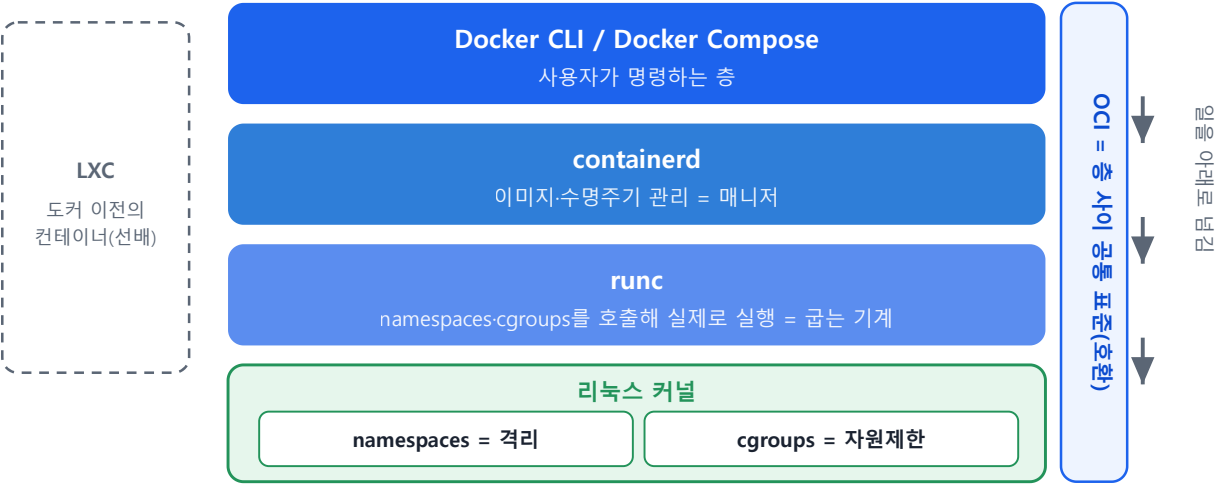
기술	한 줄 역할	비유
Docker	이미지·컨테이너를 쉽게 다루게 해주는 대표 도구	붕어빵 장사 전체 세트
namespaces	컨테이너끼리 격리(칸막이) — 리눅스 커널 기능	방마다 칸막이
cgroups	컨테이너별 CPU·메모리 상한 — 리눅스 커널 기능	방마다 전기·수도 한도
LXC	도커 이전(및 초기 도커가 기반으로 삼았던) 리눅스 컨테이너 기술	도커의 선배격
containerd	이미지 내려받기·컨테이너 수명주기를 관리하고, 실제 실행은 runc에 넘기는 런타임 매니저	주문을 받아 굽는 기계에 넘기는 매니저
runc	namespaces·cgroups를 직접 호출해 컨테이너를 실제로 생성·실행 하는 저수준 OCI 런타임	실제로 붕어빵을 굽는 기계
OCI	이미지·런타임이 지켜야 할 표준 규격 (Open Container Initiative)	붕어빵 틀 규격 표준
Docker Compose	여러 컨테이너를 파일 하나로 한 번에 정의·실행	세트 메뉴 주문서

층으로 정리하면: 위에서 아래로 **Docker(명령·Compose) → containerd(런타임 매니저) → runc(실제 실행) → 리눅스 커널(namespaces·cgroups)** 순으로 일을 넘깁니다. containerd는 "무엇을 굽을지 관리"하고, 커널 기능을 직접 불러 **실제로 컨테이너를 굽는 주체는 runc**입니다. **OCI**는 이 부품들이 서로 갈아 끼워져도 호환되도록 정한 **공통 표준**이라, 도커가 아닌 다른 도구를 써도 같은 이미지가 돌아갑니다.

Docker Compose 한마디 더: 웹서버 + 스프링부트 + DB처럼 컨테이너를 여러 개 띄워야 할 때, 하나씩 **run** 하는 대신 **docker-compose.yml** (또는 최신 표기 **compose.yaml**) 파일에 적어두고 **docker compose**

up 한 방으로 전부 띄웁니다. 비유: 여러 메뉴를 따로 주문하지 않고 "세트로 주세요" 한마디로 끝내는 것.

위에서 아래로 일을 넘긴다: Docker → containerd → runc → 커널

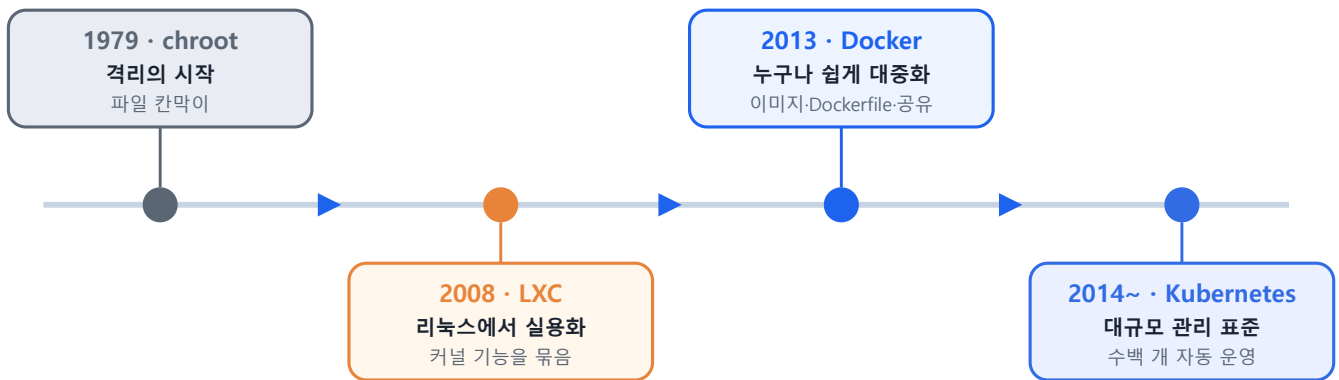


4 컨테이너 기술의 발전사

결론부터

컨테이너는 2013년에 갑자기 튀어나온 기술이 아닙니다. "여러 프로그램을 한 컴퓨터에서 서로 간섭 없이 격리(isolation)해서 돌리자"는 고민은 40년 넘게 조금씩 발전해 왔고, **Docker(도커)**는 그 오래된 기술을 '누구나 쉽게 쓸 수 있게' 포장해 대중화시킨 결정타였습니다.

큰 흐름을 한 문장으로: 격리라는 숙제 → 리눅스 커널이 격리 도구를 마련 → Docker가 그 도구를 쉽게 포장 → 컨테이너가 너무 많아지자 관리(오케스트레이션) 경쟁 → 쿠버네티스(Kubernetes)가 사실상 표준으로 승리.



각 단계는 '직전의 불편'을 풀었다 (자세한 연표는 다음 표 참고)

강의 팁

말로 설명할 땐 아래 4단계만: chroot(격리의 시작) → LXC(리눅스 실용화) → Docker(누구나 쉽게) → 쿠버네티스(대규모 관리 표준). 세부 연표 표는 나중에 읽어보는 참고용이니 강의 중 연도를 외우게 하지 마세요.

왜 '격리'가 필요했나 (출발점)

옛날에는 서버 한 대에 여러 프로그램을 같이 올렸는데, 서로 간섭하는 게 문제였습니다. A가 쓰는 라이브러리 버전과 B가 쓰는 버전이 충돌하거나, A가 B의 파일을 건드리기도 했죠. 마치 **칸막이 없는 사무실에서 여러 팀이 같은 책상과 서랍을 공유**하는 상황입니다. 그래서 "각 팀에게 독립된 방을 주자"가 컨테이너 역사 전체를 관통하는 주제입니다.

타임라인: 각 단계는 '직전의 불편'을 풀었다

시기(무렵)	기술	직전의 불편 → 해결한 것
1979	chroot (유닉스)	프로세스가 보는 최상위 폴더(/)를 가짜로 바꿔 파일 시스템 일부만 보게 만든 최초의 '방 칸막이' . 단, 원래 보안용이 아니라 빌드·테스트 환경 분리용이었고, 권한 있는 프로세스는 빠져나갈 수 있어 완전한 격리는 아니었음
2000	FreeBSD Jails	chroot는 파일만 격리했음 → 프로세스 목록·네트워크(전용 IP)까지 함께 가둔 '진짜 감옥(jail)'
2004~2005	Solaris Containers/Zones	상용 유닉스에서 자원 배분까지 포함한 격리를 제품으로 상용화 (2004 발표 → 2005 정식 출시)
네임스페이스 2002~2013 / cgroups 2008	네임스페이스 + cgroups (리눅스 커널)	리눅스 자체에 시야 격리(네임스페이스) 와 자원 제한(cgroups) 을 내장. 오늘날 컨테이너의 두 기둥 . 네임스페이스는 2002년(mount)부터 2013년(user)까지 순차 추가됐고, cgroups는 구글이 개발해 2008년 커널에 병합
2008	LXC (리눅스 컨테이너)	커널의 두 기둥을 묶어 리눅스 컨테이너를 처음 실용화. 단, 설정이 여전히 어려움
2013	Docker 공개 (dotCloud)	LXC는 전문가용이었음 → 이미지·Dockerfile·명령어 한 줄로 '누구나' 쓰게 포장
2015	OCI 출범 / 쿠버네티스 1.0	회사마다 포맷이 갈라질 위험 → 표준(OCI) 제정. 동시에 관리 도구 경쟁 본격화
2016~2017	containerd·runc 분리	Docker에 뭉쳐 있던 실행 엔진을 떼어내 표준 부품 으로 분리
2017 무렵	쿠버네티스 사실상 표준	Swarm·Mesos와의 오케스트레이션 경쟁에서 사실상 승리 (업계의 통설)
2019~	Docker 역할 재편	운영은 쿠버네티스에 넘기고, Docker는 '빌드·개발 도구'로 집중

참고

chroot가 완전한 격리가 아니었기 때문에, 그 빈틈을 메우려고 Jails(2000)·Zones(2004~2005)가 뒤이어 나왔다고 보면 흐름이 자연스럽습니다.

두 개의 기둥: 네임스페이스와 cgroups (2002~2013에 걸쳐 완성)

리눅스 컨테이너를 실제로 굴러가게 하는 진짜 엔진은 이 두 커널 기능입니다.

- **네임스페이스(namespaces) = 시야 격리**. 프로세스가 '자기 것만' 보이게 만듭니다. (비유: 옆 팀 책상이 안 보이게 하는 칸막이) 한 번에 완성된 게 아니라, **2002년 첫 mount 네임스페이스(커널 2.4.19)부터 2013년 user 네임스페이스(커널 3.8)까지** 2000년대~2010년대 초에 걸쳐 하나씩 커널에 추가됐습니다.

- **cgroups = 자원 제한.** CPU·메모리를 어디까지 쓸지 배정합니다. (비유: 팀마다 쓸 전기·책상 수를 정해 줌) 구글이 개발해 **2008년 리눅스 커널(2.6.24)에 병합**되었습니다.

이 둘이 있어야 비로소 컨테이너가 성립합니다. Docker든 다른 도구든, 바닥에서 일하는 건 결국 이 커널 기능입니다.

2013년 Docker: '어려운 기술'을 '쉬운 경험'으로

LXC로 컨테이너를 만들 수는 있었지만 설정이 복잡해 전문가의 영역이었습니다. Docker는 기술을 새로 발명한 게 아니라 **경험을 발명**했습니다.

1. **이미지(image)** 라는 표준 포장 방식
2. **Dockerfile** 로 만드는 법을 레시피처럼 기록
3. **Docker Hub** 로 이미지 공유
4. **명령어 한 줄로 실행**

비유하면, **리눅스 커널 기능은 원래 있던 밀가루와 오븐**이고, **Docker는 붕어빵 틀 + 레시피 카드 + 프랜차이즈 배포망**입니다. 재료는 있었지만, 누구나 똑같은 붕어빵을 굽게 만든 게 혁신이었죠. PaaS 회사 **dotCloud**가 사내 도구로 만든 것을 2013년 오픈소스로 공개했고, 회사 이름도 Docker로 바꿨습니다. (초기 Docker는 LXC 위에서 돌다가, 2014년 자체 실행 방식인 libcontainer로 갈아탔고, 이것이 훗날 runc로 이어집니다.)

표준화와 부품 분리 (2015~2017 무렵)

Docker가 너무 커지자 두 가지 걱정이 생겼습니다.

1. **포맷 독점 우려** → **OCI(Open Container Initiative)** 가 2015년 리눅스 재단 산하로 출범해 이미지·런타임 **표준**을 정했습니다. 덕분에 특정 회사에 종속되지 않게 됐죠.
2. **덩치 우려** → Docker 내부 엔진을 떼어내 **runc**(컨테이너를 실제로 실행), **containerd**(컨테이너의 수명 관리) 같은 표준 부품으로 분리했습니다. (비유: 만능 기계 하나를 표준 규격 부품들로 나눠 다른 회사도 조립 가능하게)

오케스트레이션 전쟁, 그리고 쿠버네티스 (2014~2017 무렵)

컨테이너 만들기가 쉬워지자 이번엔 **수백~수천 개**가 생겼습니다. "이 많은 걸 누가 자동으로 배치하고, 죽으면 살리고, 필요하면 늘리나?" 이 관리 자동화가 **오케스트레이션(orchestration)** 입니다. 3파전이 벌어졌습니다.

- **Docker Swarm** (도커가 직접)
- **쿠버네티스** (구글이 내부 시스템 Borg 경험을 바탕으로 2014년 공개, 2015년 1.0 및 CNCF 기부)
- **Apache Mesos**

2017년 무렵 쿠버네티스가 사실상 표준으로 굳어졌습니다. 이는 어떤 기구가 강제한 게 아니라 **업계가 사실상 합의한 결과**입니다. 그해 Docker까지 자사 제품에 쿠버네티스 지원을 발표했고, 주요 클라우드도 잇따라 지원을 추가했죠. 비유하면, **컨테이너가 개별 화물 컨테이너라면 오케스트레이션은 수천 개를 배치·운송하는 항구 관제 시스템**입니다.

지금의 Docker: 역할이 바뀌었다

운영 무대의 주인공은 쿠버네티스로 넘어갔지만 Docker가 사라진 건 아닙니다. **개발자가 로컬에서 이미지를 만들고 테스트하는 '빌드·개발 도구'로 자리 잡았습니다**(Docker Desktop 등). 2019년 Docker는 기업용 사업부(Docker Enterprise)를 Mirantis에 매각하고 개발자 도구에 집중했습니다.

참고로 쿠버네티스는 1.24 버전(2022년)부터 도커 전용 연결부(dockershim)를 제거했는데, "쿠버네티스가 도커를 버렸다"는 오해와 달리 **OCI 표준 덕분에 도커로 만든 이미지는 지금도 그대로 잘 돌아갑니다**.

한눈에 기억할 것

- **격리 아이디어는 1979년부터** — 컨테이너는 40년 넘는 진화의 결과다.
- **진짜 엔진은 리눅스 커널의 네임스페이스 + cgroups** — Docker는 그걸 쉽게 포장했을 뿐.
- **Docker의 혁신은 기술이 아니라 '경험'** (이미지·Dockerfile·공유).
- **운영은 쿠버네티스, 빌드는 Docker** — 이것이 오늘날의 역할 분담이다.

5 쿠버네티스(Kubernetes) 기초

결론부터: 컨테이너가 몇 개일 때는 사람이 손으로 켜고 끌 수 있습니다. 하지만 수십~수백 개가 되면 손으로는 불가능합니다. **쿠버네티스는 이 많은 컨테이너를 사람 대신 자동으로 관리해 주는 "컨테이너 관리 시스템"** 입니다. 앞에서 배운 도커가 "컨테이너 한 척"을 다루는 법이었다면, 쿠버네티스는 "수백 척의 배를 항구에서 통제하는 관제탑"에 가깝습니다.

1. 왜 필요한가 — 컨테이너가 많아지면 손으로는 못 한다

컨테이너 하나를 띄우는 것은 도커 명령 한 줄이면 됩니다. 그런데 서비스가 커져서 컨테이너가 100개가 되면, 아래 일들을 **사람이 24시간 지켜보며** 해야 합니다.

1. **죽으면 다시 살리기** — 컨테이너 하나가 새벽 3시에 죽으면? 누군가 깨어나 다시 켜야 합니다.
2. **바쁘면 늘리고, 한가하면 줄이기(스케일)** — 접속자가 몰리면 컨테이너를 5개에서 20개로 늘리고, 새벽엔 다시 줄여야 합니다.
3. **무중단 배포** — 새 버전을 올릴 때 서비스를 멈추지 않고, 낡은 컨테이너를 새것으로 하나씩 조용히 교체해야 합니다.
4. **서로 찾고 연결하기(네트워크)** — 컨테이너는 죽고 다시 뜰 때마다 주소(IP)가 바뀝니다. "결제 컨테이너"가 "회원 컨테이너"를 매번 어떻게 찾을까요?

이걸 사람이 다 하는 것은 불가능합니다. 그래서 등장한 것이 **오케스트레이션(Orchestration, 지휘·조율)** 입니다. 오케스트라에서 수십 명의 연주자를 지휘자 한 명이 맞추듯, **여러 컨테이너의 배치·복구·확장·연결을 자동으로 조율하는 것**을 컨테이너 오케스트레이션이라 하고, 그 대표 도구가 쿠버네티스입니다.

2. 핵심 개념 5가지

낯선 용어가 많지만, "물류 창고" 하나로 묶어서 보면 쉽습니다.

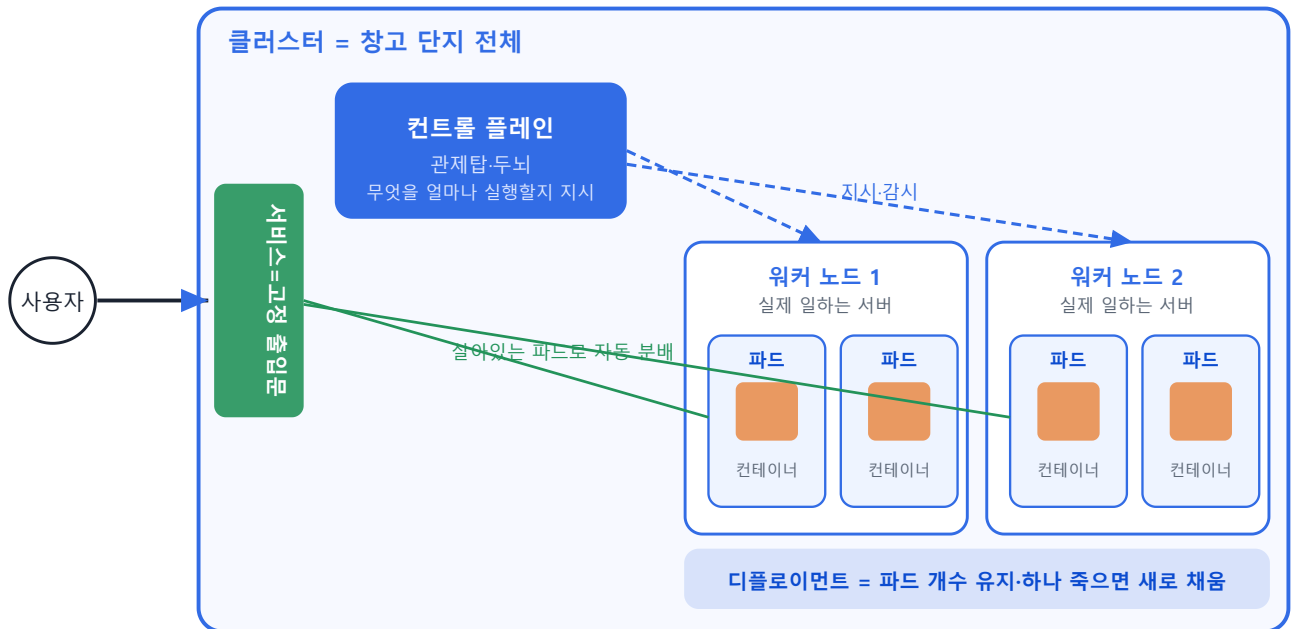
용어	한 줄 정의	비유
클러스터(Cluster)	쿠버네티스가 관리하는 전체 컴퓨터 묶음	물류 창고 단지 전체
노드(Node)	컨테이너가 실제로 실행되는 서버 한 대	창고 건물 한 채
파드(Pod)	배포의 최소 단위 (컨테이너 1개 이상을 담는 상자)	팔레트(짐 받침대) 한 장
디플로이먼트(Deployment)	"파드 몇 개를 유지할지" 관리·교체 담당	창고 관리자
서비스(Service)	파드로 들어가는 안정적인 출입문(주소)	창고 정문·안내 데스크

조금 더 풀어 보면:

- **노드에는 두 종류가 있습니다.** **컨트롤 플레인(Control Plane, 옛 명칭 마스터/Master)** 은 "무엇을 얼마나 실행할지 결정하고 지시하는 두뇌"이고, **워커 노드(Worker Node)** 는 "실제로 컨테이너를 돌리는 일꾼"입니다. 관제탑(컨트롤 플레인) 하나가 여러 작업장(워커)에게 일을 시키는 구조입니다.
- **파드는 컨테이너를 직접 다루지 않고 한 겹 감싼 상자**입니다. 보통 파드 하나에 컨테이너 하나지만, 딱 붙어 함께 움직여야 하는 보조 컨테이너를 같이 담기도 합니다. 이때 **같은 파드에 담긴 컨테이너들은**

같은 주소(IP)와 저장 공간을 공유하며 한 몸처럼 움직입니다. 쿠버네티스는 컨테이너가 아니라 **파드** 단위로 배치·복구합니다.

- **서비스가 필요한 이유**는 파드의 주소가 계속 바뀌기 때문입니다. 파드가 죽고 새로 떠서 IP가 바뀌어도, 서비스라는 **고정된 출입문**이 알아서 살아있는 파드로 연결해 줍니다. 손님은 창고 정문 주소만 알면 되고, 안쪽 짐이 어디로 옮겨졌는지는 몰라도 됩니다.



3. 선언적 관리(Declarative) — 이것이 핵심 발상

쿠버네티스가 특별한 이유는 "어떻게(how) 하라"를 일일이 지시하는 대신 "무엇을 원하는지(what)"를 선언한다는 점입니다.

- **명령형(Imperative):** "컨테이너를 켜라 → 죽었으니 다시 켜라 → 하나 더 켜라" 처럼 **매 순간 사람이 지시**합니다.
- **선언형(Declarative):** "이 앱은 항상 3개 유지해줘" 라고 원하는 **상태(Desired State, 원하는 상태)만 선언**합니다. 그러면 쿠버네티스가 현재 상태를 계속 확인하다가, 2개로 줄면 알아서 1개를 더 띄우고, 4개가 되면 1개를 없앱니다. 이 선언형이 쿠버네티스의 **기본이자 권장 방식**입니다. (하나씩 직접 지시하는 명령형 명령어도 지원하긴 하지만, 자동 복구·유지의 힘은 선언형에서 나옵니다.)

```
replicas: 3 # "파드를 항상 3개 유지해줘" - 방법이 아니라 목표만 선언
```

이것은 **온도 조절기(에어컨)와 같습니다**. 우리는 "22도로 맞춰줘"라고 목표만 말하지, "지금 켜라, 이제 꺼라"를 일일이 지시하지 않습니다. 온도가 오르면 알아서 켜고 내려가면 끕니다. 쿠버네티스도 목표 상태와 현재 상태의 **차이를 끊임없이 메우는** 방식으로 동작하고, 이 덕분에 새벽에 컨테이너가 죽어도 사람 없이 스스로 복구됩니다.

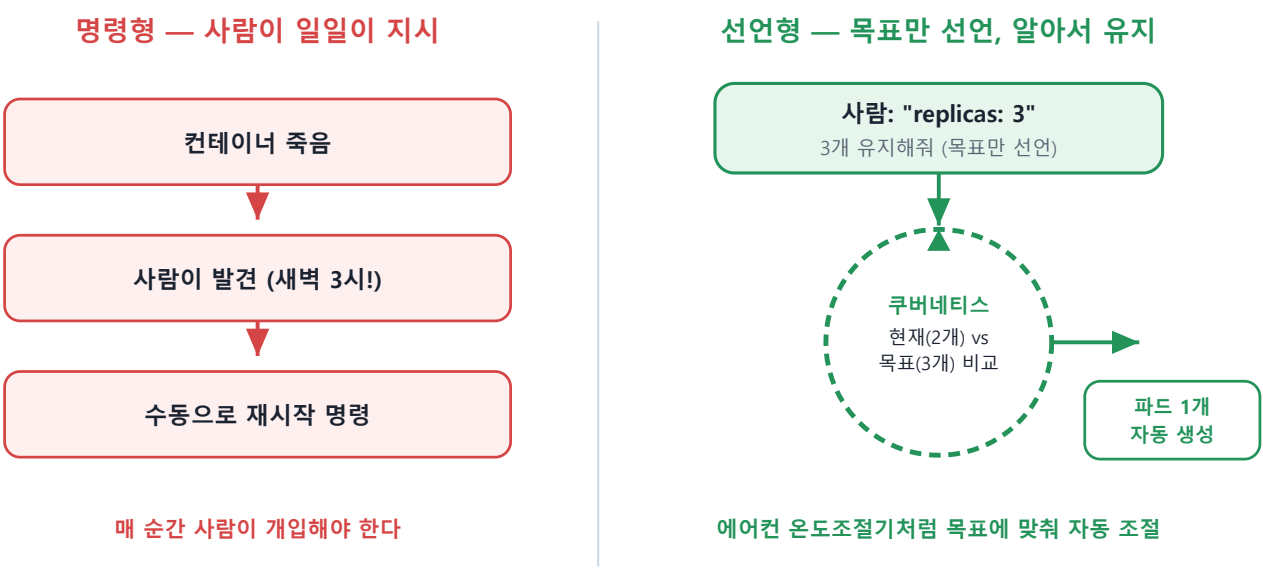
4. 균형 잡힌 시각 — 항상 정답은 아니다

쿠버네티스는 강력하지만 **무겁고 배우기 어렵습니다**. 개념도 많고, 작은 규모에는 관리 부담이 오히려 더 큽니다. **컨테이너 한두 개짜리 개인 프로젝트나 소규모 서비스에는 과한 선택**일 수 있고, 이럴 땐 도커만

으로도 충분한 경우가 많습니다. "많은 컨테이너를 안정적으로 계속 굴려야 할 때" 진가를 발휘하는 도구라고 이해하면 됩니다.

5. 이름의 유래 (가볍게)

Kubernetes는 그리스어 **κυβερνήτης(조타수·키잡이)**에서 왔습니다. 컨테이너(화물)를 실은 배를 안전하게 조종하는 사람이라는 뜻이니, 하는 일과 잘 맞는 이름입니다. 흔히 **k8s**로 줄여 쓰는데, 이는 첫 글자 **K**와 끝 글자 **s** 사이에 글자가 **8개(ubernete)** 있다는 뜻입니다. 참고로 원래 구글이 만들어 **2014년** 오픈소스로 공개했고, **2015년**에는 특정 회사에 속하지 않는 중립 재단인 **CNCF(클라우드 네이티브 컴퓨팅 재단)**로 이관되어 지금은 여러 회사가 함께 발전시키고 있습니다.



마무리 — 오늘 기억할 3가지

핵심 요약

① 컨테이너 = 환경째 포장한 상자. "내 컴퓨터에선 되는데" 문제를 크게 줄인다. (VM보다 가벼운 이유 = 게스트 OS 없이 커널 공유)

② 이미지 = 봉어빵 틀, 컨테이너 = 봉어빵. Docker로 이미지를 만들고, 레지스트리(Docker Hub)로 공유 한다. (build → push/pull → run)

③ 쿠버네티스 = 많은 컨테이너를 자동 관리하는 관제탑. "3개 유지해줘"라고 선언 하면 죽어도 알아서 복구한다(선언형).

Docker와 쿠버네티스는 '이미지'로 이어진다

Docker까지 배우면 내 앱을 이미지로 굳혀 컨테이너 한 개로 어디서나 똑같이 띄우는 데까지 완성됩니다. 문제는 서비스가 커지면 이 컨테이너가 여러 서버에 수십~수백 개로 흩어진다는 점입니다. 죽으면 되살리고, 물리면 늘리고, 무중단으로 교체하고, 주소가 바뀌어도 서로 찾게 하는 일을 사람이 24시간 할 수는 없습니다.

한 줄기로

내 app.jar → (Docker) 이미지 빌드 → 레지스트리 push → (쿠버네티스) pull → 파드로 실행·복구·확장. 즉 Docker는 '한 개를 만들어 포장하는 도구', 쿠버네티스는 '그 포장물을 대규모로 운영하는 관제탑'입니다. 같은 이미지를 매개로 둘이 한 줄기로 이어집니다.

한 장 요약 — 핵심 용어

컨테이너	앱 + 실행 환경을 통째로 담은 격리된 실행 단위 (VM보다 가벼움)
이미지	컨테이너의 '봉어빵 틀' — 읽기전용 설계도. Dockerfile로 build
레지스트리	이미지의 깃허브(Docker Hub). push(올리기)/pull(내려받기)
VM	게스트 OS를 통째로 올리는 방식 — 무겁지만 격리가 강함
오케스트레이션	많은 컨테이너의 배치·복구·확장·연결을 자동 조율
파드(Pod)	쿠버네티스 배포의 최소 단위(컨테이너를 감싼 상자)
디플로이먼트	"파드 몇 개 유지"를 관리·교체(죽으면 새로 채움)
서비스(Service)	파드로 들어가는 고정된 출입문(주소) — IP가 바뀌어도 연결 유지
선언형	"원하는 상태(3개 유지)"만 선언하면 시스템이 알아서 맞춤

균형 잡기

쿠버네티스는 강력하지만 **무겁고 배우기 어렵습니다**. 컨테이너 한두 개짜리 작은 프로젝트엔 오히려 과할 수 있어, 그럴 땐 Docker(또는 Docker Compose)만으로 충분합니다. "많은 컨테이너를 안정적으로 계속 굴려야 할 때" 진가를 발휘하는 도구입니다.

다음 단계 (실습 예고)

- **Docker 실습:** `docker run hello-world` 로 첫 컨테이너 → 스프링부트 앱을 Dockerfile로 빌드해 실행
- **여러 컨테이너:** `docker compose up` 으로 앱 + DB를 한 번에
- **쿠버네티스 맛보기:** `minikube` (내 PC에 미니 쿠버네티스)로 파드·디플로이먼트·서비스 직접 띄워보기

이 자료는 여러 단계의 초안 작성 → 기술 사실 검증(팩트체크) → 교정 과정을 거쳐 작성되었습니다. 다만 컨테이너 생태계는 빠르게 바뀌고 일부 연도는 '무렵'으로 표기했으니, 정확한 최신 정보는 공식 문서(docker.com, kubernetes.io)에서 확인하세요.

대상: 인턴·초급 개발자 · 30분 강의용 참고자료